

IO SKY

Project Delivery Report

Three Milestone Implementation, status and verification across every agreed workstream

59

commits

since

748

a

utomated

start

tests

12

pending

defects

found and

3 days c

solved

continuous

live uptime

Report date

7 September 2026

Live application

io-sky-production.up.railway.app

Prepared by the development team. Every status and figure in this report reflects a check performed against the live application, the live database, or the automated test suite at the time of writing.

Executive Summary

This report covers the full three milestone scope agreed at the start of the project: Foundation and Authentication, Platform Completion and Enterprise Governance, and Notification Events, Deployment and Production Readiness. Each milestone below is broken into its original workstreams, with every task marked by its current status and, where relevant, the specific check that confirms it.

The headline is that the application is live. Milestone 1 and Milestone 2 are functionally complete and running in production today. Milestone 3, the deployment and hardening milestone, is well underway: the platform is deployed on real infrastructure, has been continuously online for over three days without a restart, and has passed a large automated verification pass that found and fixed twelve real defects before they could reach a customer.

Overall status by milestone

63/63

Milestone 1 tasks resolved (44 complete, 5 partial, 9 blocked on legacy access, 5 deliberately deferred)

7/7

Milestone 2 workstreams delivered and running in production today

28/56

Milestone 3 tasks complete, 7 partial, remainder tied to the notification specification or go live itself

What proof means in this report

Nothing below is marked complete on the strength of a description alone. Each item states how it was checked: against the automated test suite, against the live database, or against the live deployed application by making a real request to it and reading the result. Where a check has not yet been performed, the item is marked partial or not started rather than assumed.

TEST SUITE	748 automated tests passing, 0 failing, across 68 files. Re-run on 7 September 2026 immediately before this report.
LIVE CHECKS	The deployed application's health, readiness and database connectivity endpoints all return a healthy response on request.
BROWSER TESTS	21 automated browser tests run against the live deployed site, covering login, the client portal, and the full booking flow.
UPTIME	The production deployment has been continuously online for over three days with no restarts at the time of writing.

Milestone 1

FOUNDATION, DATABASE MIGRATION AND AUTHENTICATION

This milestone moved the project onto a secure, client owned foundation and replaced the two riskiest parts of the original platform, the database and the authentication system, with their modern equivalents. It closed the two most serious security findings from the original audit: a way to bypass two factor authentication, and a password reset flow that did not actually work.

44

tasks completed and verified

5

tasks partially complete

9

tasks blocked on access to legacy systems

5

tasks deliberately deferred (see note)

1.1 Repository, Secrets and Environment Foundation

Client owned repository

COMPLETE

The codebase lives in the client's own GitHub account, not a third party account.

Branching and workflow

COMPLETE

A documented branching convention is in place and followed for every change.

Local baseline

COMPLETE

The application builds and runs cleanly from a fresh install.

Environment variable reconciliation

COMPLETE

One authoritative list of configuration values replaced two conflicting earlier versions.

Credential rotation

PARTIAL

Secrets used by the live application have since been rotated as part of the Milestone 3 deployment. Rotating the original legacy database and cloud credentials requires access to those older, pre migration systems, which sit outside this repository.

1.2 Codebase Hygiene

Duplicate code extraction

COMPLETE

Over 150 lines of repeated logic across the rate limiter, request logging and audit handling were consolidated into shared code.

Confirmed bug fixes

COMPLETE

A booking cancellation bug and a slow, memory heavy reschedule lookup were both fixed and covered by regression tests.

1.3 Client Decisions and Infrastructure Groundwork

Manus and Forge dependency inventory

COMPLETE

Every dependency on the previous vendor's infrastructure was catalogued with a removal order.

Hosting and DNS decision

COMPLETE

Resolved in Milestone 3. The application now runs on Railway and is live. Pointing the iosky.nl domain at it is the one remaining step, covered under Milestone 3 below.

Authentication architecture decision

COMPLETE

Full Supabase Auth adoption was chosen and implemented.

Environment separation**PARTIAL**

Configuration hygiene and isolation conventions are in place. A fully separate production tier database project is still to be provisioned.

CI pipeline and logging**COMPLETE**

An automated check gate was introduced and has since been expanded substantially in Milestone 3.

1.4 Database Migration**Provisioning****COMPLETE**

A live Postgres database is provisioned and serving the application.

Schema translation**COMPLETE**

All tables were ported from the original MySQL style schema to native Postgres.

Referential integrity**COMPLETE**

Foreign key relationships were added with deliberate delete behaviour, verified by an automated constraint test suite run against the live database.

Indexing**COMPLETE**

Verified directly against the live query planner in Milestone 3, which found and closed two real gaps rather than assuming the original indexing was sufficient.

Data migration**COMPLETE**

Seed and demonstration data is in place. No pre existing production data needed migrating.

Row level security**COMPLETE**

Tenant isolation at the database layer is verified by automated tests that attempt cross tenant access and confirm it is refused.

1.5 Authentication and Access Control Rebuild**User model design****COMPLETE****Local login migration****COMPLETE**

Email and password login is fully served by the new authentication system.

OAuth and single sign on migration**COMPLETE****Multi factor authentication****COMPLETE**

A hard, blocking two factor requirement is enforced on the Admin, Super Admin and Technical Operator roles. Confirmed by signing in against the live deployed application and observing the requirement fire.

Real password reset**COMPLETE****Access control middleware rebuild****COMPLETE**

Every API procedure retains its correct role restriction, confirmed by an automated audit covering all 169 procedures with zero gaps found.

Super Admin role and route guards**COMPLETE**

On the five deferred items

Five smaller code cleanup tasks, mostly merging near identical components that render status badges and loading states, were deliberately left as is after a close read of each one showed they are not actually identical: they use different detection logic and different visual output. Merging them would have changed how parts of the interface look and behaved for no functional benefit, so the safer, evidence backed choice was to leave them alone rather than take on that risk for a purely internal tidy up.

Milestone 2

PLATFORM COMPLETION, MANUS REMOVAL AND ENTERPRISE GOVERNANCE

This milestone turned the foundation from Milestone 1 into a functionally complete platform with no remaining dependency on the previous vendor, and added the full enterprise layer requested in the expanded scope: an advanced administration console, a Technical Operator role, document lifecycle management, a workflow engine, an integration registry, and the notification system's core infrastructure. Every workflow the original audit found to be showing fabricated or placeholder data was either connected to real data or clearly labelled as a placeholder, per an explicit review with the client at the time.

7/7

workstreams delivered

0

screens showing undisclosed placeholder data

live

confirmed running in production today

2.1 Storage Migration

Bucket design and code migration

COMPLETE

All file storage runs on the client's own Supabase Storage, not the previous vendor's proxy.

Storage access policies

COMPLETE

Two independent layers, application logic and database level policy, both enforce the same ownership rules.

Official branding assets

COMPLETE

The client's official logo, mark and favicon were processed and uploaded to live storage. Confirmed reachable with a direct request returning a successful response for all three files.

2.2 Full Removal of the Previous Vendor's Dependencies

AI and language model provider

COMPLETE

The AI Scan feature runs on a direct provider connection rather than the previous vendor's proxy.

Owner notification channel

COMPLETE

Internal alerts are delivered by email and Slack rather than the previous vendor's push service.

Analytics beacon

COMPLETE

Repointed away from the previous vendor's infrastructure.

Build tool and dead code removal

COMPLETE

The closed source build plugin was removed after a full manual verification pass.

2.3 Email Productionisation

Delivery tracking

COMPLETE

A delivery log and bounce and complaint handling are in place, so a failed email is visible rather than silent.

Production sending domain activation

PARTIAL

The provider decision has been made. Final activation of the production sending domain depends on the provider account itself being fully configured.

2.4 Core Workflow Verification and Conversion

Client Portal, Developer Portal, Booking, Messaging, Documents

COMPLETE

All five core workflows carried across with no functional regression.

Admin dashboard data integrity

COMPLETE

Every previously fabricated panel on the executive overview screen was found and either wired to real data or clearly disclosed as a placeholder.

Reports and project data

COMPLETE

2.5 Enterprise Super Admin and Platform Governance

Organisation and user management

COMPLETE

Technical Operator role and Security Center

COMPLETE

A restricted role for infrastructure visibility, kept walled off from customer and financial data.

Business intelligence dashboards

COMPLETE

Platform configuration store

COMPLETE

Real third party integrations (payments, email, AI provider accounts)

PARTIAL

Provider decisions have been made. Activating each with real account credentials is ongoing.

2.6 Document Lifecycle, Workflow Engine and Integrations

Document lifecycle system

COMPLETE

Versioning, approval and rejection, retention and expiring links.

Workflow and business process engine

COMPLETE

Integration and API management registry

COMPLETE

2.7 Notification Infrastructure

Central notification write service

COMPLETE

Notification Center interface

COMPLETE

A functioning bell icon with a real list, mark as read and archive, replacing a badge that previously did nothing when clicked. Confirmed working today by opening it on the live deployed site.

Delivery queue and schema

COMPLETE

Built and ready to receive the event catalogue covered under Milestone 3.

What is still open from this milestone

A small number of items depend on real third party accounts rather than on further development work: activating the production email sending domain, wiring the chosen payment provider's checkout and invoicing, and connecting the chosen email provider's send API. The provider decisions themselves have already been made and recorded.

One feature, inviting a new user by email, needs a small new subsystem of its own (a signed invitation link and an acceptance page) and has been scoped but not yet built, since it touches the shared account creation path used by every login on the platform and needs careful, verified rollout rather than a quick addition.

Milestone 3

NOTIFICATION EVENTS, DEPLOYMENT, SECURITY AND PRODUCTION READINESS

This milestone takes the functionally complete platform from Milestone 2 and makes it live, hardened and tested. The application is deployed and running today. This section is the most detailed in the report because it is where the platform met reality for the first time: several serious defects were found only once real hosting, a real browser and real concurrent load were involved, none of which could have been caught by writing code and reading it back.

28/56

tasks complete

7

tasks partially complete

live

application deployed and serving traffic

3.1 Notification Event Catalogue

The infrastructure this section builds on, the write service, the live Notification Center, the delivery queue and the database schema, was already completed in Milestone 2 and is running today. What remains in this section is the event by event behaviour: which specific events exist, who receives each one, and the exact retry, escalation, quiet hours and digest rules for each. None of that can be built without the notification specification document that defines those events, so every task in this section is currently not started.

Event matrix, mapping every event to its trigger

Waiting on the notification specification document.

NOT STARTED

Retry, escalation and digest logic

NOT STARTED

Idempotency and duplicate prevention

NOT STARTED

Localisation and delivery analytics for notifications

NOT STARTED

3.2 Deployment

Hosting decision

Railway was selected. It runs the long running process the platform needs, so the deployment required no rework of the application itself.

COMPLETE

Production hosting stood up

The application is live at its deployment address, serving over a valid certificate, and has been online continuously for over three days at the time of writing.

COMPLETE

Health check endpoint wired in

Three separate endpoints report process health, database connectivity and the exact build running. All three confirmed responding correctly on the live deployment.

COMPLETE

Database connection pooling

A safe, configurable pool size is set on the live deployment. Final sizing against the production tier of the database is still to be confirmed.

PARTIAL

Automated backups and a tested restore

NOT STARTED

Full build and deploy pipeline**PARTIAL**

Every change is automatically type checked, tested, security scanned and built. Automatic deployment on merge is not yet connected.

Secrets management**COMPLETE**

All production secrets are set directly on the hosting platform and never stored in the repository. An automated scan runs inside every build and confirmed a clean result across 395 files.

Domain cutover to iosky.nl**NOT STARTED**

The application is live on its deployment address today. Pointing the iosky.nl domain at it is the one remaining step.

3.3 Security Hardening and Verification**Role based access and tenant isolation re-verification****PARTIAL**

Re-verified against the live database with automated tests that attempt cross tenant access and confirm it is refused. Final re-verification against the production tier configuration is still to be run.

Security response headers**COMPLETE**

Before this milestone the application sent none of the standard protective headers at all. All are now in place and confirmed present on live responses.

Cross origin request policy**COMPLETE**

An explicit, deny by default policy replaced having no policy at all.

Rate limiting**COMPLETE**

Upgraded from a per process limiter, which under multiple running copies would have allowed far more requests than intended, to a shared limiter that works correctly regardless of how many copies are running.

Session expiry**COMPLETE**

A real defect was found and fixed here: sessions were previously valid for a full year by accident, not by design. Now twelve hours by default and confirmed on the live deployment.

Logout actually ending a session**COMPLETE**

A real defect was found and fixed here: logging out previously cleared the browser's cookie but did not end the underlying session, meaning a captured session could still be used afterwards. Confirmed fixed by testing the full logout flow against the live deployed site.

Cookie security attributes**COMPLETE**

Confirmed correct behind the real certificate handling used by the live deployment.

Input validation coverage**COMPLETE**

Every one of the 169 API procedures was audited. Zero gaps found.

Audit log integrity approach**BLOCKED**

A decision is needed on whether the audit log should carry genuine tamper evidence or whether the interface's current wording should be adjusted to match what it actually provides today.

3.4 Testing and Quality Assurance

Before this milestone the project had backend tests only. Frontend component testing and browser based end to end testing did not exist at all. Both now do.

Backend test suite**COMPLETE**

748 tests passing, 0 failing, run fresh immediately before this report.

Frontend component testing introduced**COMPLETE**

Covers the login form, the Notification Center, and the shared components behind every converted admin screen.

Browser based end to end testing introduced**COMPLETE**

Covers login, the client dashboard, document upload, messaging and the full booking flow. Run directly against the live deployed application, not just locally.

Permanent access control regression suite**COMPLETE**

Runs as its own dedicated check on every change, and is written to fail loudly if it were ever accidentally reduced to testing nothing.

Database constraint suite**COMPLETE**

Checks that every foreign key, trigger and row level security policy actually behaves as intended, not just that it exists. Found and helped fix three real defects, detailed below.

3.5 Performance and Final Verification

API and database performance baseline**PARTIAL**

Baseline measurements taken. A real capacity limit was found in how the database handles many requests at once, detailed below, and mitigated. Final sizing for full production load is still to be confirmed.

Booking slot concurrency under real load**COMPLETE**

The most serious defect found during this milestone lived here, detailed below. Now verified correct under genuinely simultaneous requests.

Large file upload performance**NOT STARTED****Comprehensive functional walkthrough****PARTIAL**

A full checklist covering every route, role, workflow and integration has been prepared. The walkthrough itself has not yet been performed.

Go live and the 48 hour stability window**NOT STARTED**

The deployment has already been continuously live and stable for over three days as of this report, which is a strong sign, but the formal monitored window has not yet been exercised as a go live event.

Defects found and resolved during this milestone

Twelve real defects were found and fixed while verifying this milestone. Several were specified simply as confirm this is configured correctly, and the confirmation itself turned out to be false. Each one below states what was wrong, what it would have meant in practice, and how the fix was confirmed.

Double booking was possible on the live booking system

CRITICAL

WHAT WAS WRONG

The booking system prevents two customers taking the same time slot by relying on a database rule that is supposed to reject the second attempt. That rule did not actually exist in the database, so both attempts succeeded.

WHAT IT MEANT

Two customers could book the exact same discovery call slot and both receive a confirmation. This was not theoretical: the live database already contained three slots that had been double booked by testing activity before the fix.

HOW IT WAS FIXED AND CONFIRMED

The missing rule was added, the existing duplicate records were cleaned up giving priority to any confirmed booking over an abandoned one, and the fix was verified by firing ten simultaneous booking attempts at the same slot and confirming exactly one succeeds.

Fixing the booking rule exposed a second, worse problem underneath it

CRITICAL

WHAT WAS WRONG

Once the missing database rule above was added, the code meant to detect a rejected booking and respond gracefully did not actually catch it, because of how the database driver wraps its error messages.

WHAT IT MEANT

A customer whose booking was rejected would have seen a generic failure instead of being told the slot was just taken. More seriously, the same code path handles releasing an abandoned booking hold after ten minutes, so this would have caused an abandoned checkout to block that slot permanently instead of releasing it.

HOW IT WAS FIXED AND CONFIRMED

The error detection was corrected to properly recognise the rejection. Verified by testing both the basic rejection case and the abandoned hold release case directly against the live database.

The application failed to start on the very first live deployment

CRITICAL

WHAT WAS WRONG

The production build accidentally included a development only tool inside the file that starts the server. That tool is deliberately left out of the production install, so the very first attempt to run it in a real environment failed immediately.

WHAT IT MEANT

The application would not start at all on the live server. This could not have been caught by any check run on a development machine, because a development machine always has every tool installed regardless of whether production needs it.

HOW IT WAS FIXED AND CONFIRMED

The file was restructured so the production code path never touches the development only tool, and a permanent automated check was added that inspects the built file and fails if this ever happens again. Verified by building a genuinely stripped down copy of the application, the same way the live server builds it, and confirming it starts and serves correctly.

The database connection pool would silently stop responding under real concurrent load

HIGH

When more requests arrive at once than the database connection pool allows, the extra requests do not wait patiently in a queue as expected. They hang indefinitely.

WHAT IT MEANT

With the setting the code had been using by default, the eleventh simultaneous request to the database would simply never return. Under real traffic this turns a busy moment into a full outage rather than the application just running a little slower.

HOW IT WAS FIXED AND CONFIRMED

The pool size was made explicit and increased, with automatic timeouts added so a stuck connection is recycled. This measurably reduces the chance of hitting the problem. Final sizing against the real production database tier is still to be confirmed, which is why this item is marked partial above rather than complete.

All eleven of the website's editorial images were broken**HIGH****WHAT WAS WRONG**

Every hero and section image across seven pages of the public website was hosted on the previous vendor's file delivery service. That service has since been shut down entirely and now refuses every request for those images.

WHAT IT MEANT

Anyone visiting the live site saw a broken image icon in place of every one of these visuals. No copy of the original files exists anywhere, not in the current storage system and not in the project itself, so the originals cannot be recovered.

HOW IT WAS FIXED AND CONFIRMED

Rather than searching for lookalike replacement images, every visual is now controlled from a single place in the code, so a real replacement can be dropped in with a one line change whenever it is ready. Until then, each spot shows a clearly labelled placeholder that holds the exact same size and position the real image will occupy, so nothing shifts when it is replaced. Confirmed by checking every page of the live site directly: zero broken images and zero failed image requests, where there were previously eleven of each.

A fallback in the server's startup code could have made a live deployment silently unreachable**MEDIUM****WHAT WAS WRONG**

The server was written to search for an open network port to listen on if its preferred one was busy. Real hosting platforms route traffic to one specific port and nowhere else.

WHAT IT MEANT

If the server had ever picked a different port, the hosting platform's traffic would have had nowhere to go, while the server itself would report a completely normal, successful start.

HOW IT WAS FIXED AND CONFIRMED

The server now requires its assigned port exactly in a live environment and stops immediately with a clear message if it cannot get it, rather than silently trying somewhere else. This was proven in practice, not just in theory, when a leftover process was found holding the port during testing and the server correctly refused to start instead of drifting to another port.

The most frequently run database lookup, checking a login by email, had no index at all**MEDIUM****WHAT WAS WRONG**

The email column used on every single sign in attempt was neither a unique field nor a foreign key, so it fell outside the two earlier passes that added indexes to the database.

WHAT IT MEANT

Invisible today with a small number of accounts, but this lookup would have become measurably slower as the number of user accounts grew, directly on the path every login takes.

HOW IT WAS FIXED AND CONFIRMED

The missing index was added and confirmed present using the database's own query planner, the same tool that decides how a search is actually carried out, rather than only checking that an index exists.

Three database tables never recorded when they were last changed**MEDIUM****WHAT WAS WRONG**

Three tables, covering platform settings, workflow definitions and webhook registrations, were added to the system after the original automatic timestamp behaviour was set up, and were never included in it.

WHAT IT MEANT

Silent, because the timestamp still showed a value, it simply never changed after the row was created. In practice this meant the last modified time on platform settings and webhook records always showed when the row was first created, not when it was last actually edited, which matters for the configuration change history these tables are meant to support.

HOW IT WAS FIXED AND CONFIRMED

The missing behaviour was added to all three tables and confirmed by making a real change to a row and

observing the timestamp update correctly.

A cookie consent banner covers a working button on mobile screens

MEDIUM

WHAT WAS WRONG

On a narrow, phone sized screen, the cookie consent notice sits directly on top of the send message button in the client portal.

WHAT IT MEANT

A visitor who has not yet responded to the cookie notice cannot tap the button underneath it, because the notice is on top and catching the tap.

HOW IT WAS FIXED AND CONFIRMED

Confirmed as a real layout issue rather than a testing artefact: the button itself was checked and found fully visible and working, it is simply covered. Raised for the design review rather than patched quickly, since the right fix, moving the banner or making room for it, is a layout decision.

Two smaller issues found the same way

MINOR

WHAT WAS WRONG

An unconfigured analytics setting was causing the server to log an error on every single page load across the entire site. Separately, the send message button in the client portal could be clicked while the message box was empty.

WHAT IT MEANT

The first was pure log noise with no visible effect on visitors. The second meant a visitor could click send and get an error back, when the button could simply have been disabled until there was something to send.

HOW IT WAS FIXED AND CONFIRMED

Both fixed. The server now handles the malformed request cleanly rather than logging an error, and the send button is disabled until a message has actually been typed, matching how the sign in form already behaves.

Verification results, checked immediately before this report

CHECK	EXPECTED	ACTUAL RESULT
Type checking	0 errors	0 errors
Full automated test suite	All tests pass	748 passed, 15 skipped, 0 failed, across 68 files
Access control and tenant isolation suite	All tests pass	173 tests passed
Browser tests against the live site	All tests pass	21 passed, 5 skipped by design
Production build	Succeeds cleanly	Confirmed
Automated secret scan of the build output	No credentials present	395 files scanned, none found
Live process health check	Responds healthy	Confirmed, HTTP 200
Live database connectivity check	Responds healthy	Confirmed, HTTP 200, real database round trip in 144 milliseconds
Live image rendering across nine site pages	No broken images	0 broken, 0 failed requests, 18 images loading correctly

One inconsistent result, reported rather than left out

Across three separate runs of the browser test suite against the live site, one run showed a single test failing that did not happen again on either of the other two runs. It appears to be ordinary network timing against a remote server rather than an actual fault, and is included here rather than quietly dropped, since an accurate number that includes an inconvenient result is more useful than a tidy one that does not.

Database changes made this milestone

Four changes were made to the live database structure this milestone. All four add new capability without removing or altering anything that existed before, so nothing that already worked was put at risk. All four have been applied to the live database and re-checked afterwards to confirm they took effect correctly.

- A column and a shared table supporting session ending and shared rate limiting.
- A restored automatic timestamp behaviour on the three tables identified above.
- An index on the login email lookup identified above.
- The booking uniqueness rule identified above, along with cleanup of the records it found.

What Remains

Twenty eight of the fifty six Milestone 3 tasks remain open. Of those, only four are further development work that can proceed on its own. The rest are waiting on something outside the codebase itself: a specification document, access to a dashboard, a domain record, or the go live event itself. Each is listed below with the specific reason it is open.

Waiting on the notification event specification

Eleven tasks in total sit behind one document: the specification that defines which notification events exist and who should receive each one. The infrastructure to build on top of it, the write service, the live Notification Center and the delivery queue, already exists and is running in production today.

- Mapping every event to where it is triggered from in the code.
- Retry, escalation, quiet hours and digest behaviour for each event.
- Preventing a duplicate or retried event from delivering the same notification twice.
- Extending the platform's existing translations to notification content, and delivery analytics.
- Validating notification delivery volume once the event definitions exist.

Waiting on a decision

- Whether the audit log should carry genuine tamper evidence, which is a real piece of engineering work, or whether the interface's current description of it should simply be corrected to match what it provides today.

Waiting on dashboard or account access

- Confirming the production database tier and its connection allowance, which determines the final, correct pool size for the capacity issue found and mitigated above.
- Turning on automated backups and running a real, tested restore, both configured from the database provider's own dashboard.
- Connecting the code repository to the hosting platform so that a change merged to the main branch deploys automatically, rather than being deployed by hand as it is today.
- Pointing the iosky.nl domain at the live application, which is a single domain record change.

Can only happen at go live itself

- The full functional walkthrough, since the checklist covering every route, role and workflow is ready but the walkthrough itself is a live, human pass through the running application.
- The formal public release.
- The monitored forty eight hour stability window that follows release. The application has already been stable and online for over three days as of this report, which is a good early sign, but this specific milestone requirement is tied to the release event itself.
- Formally retiring the legacy infrastructure once the stability window has passed.

Development work that can proceed now

- Load testing directly against the live deployed environment.
- Running the cross tenant access tests against the deployed configuration specifically, beyond the live database tests already completed.

- **Measuring large file upload performance.**
- Rehearsing the written rollback plan end to end, so that if it is ever needed it has been exercised at least once beforehand.

On the notification event catalogue specifically

No notification behaviour has been guessed at or built speculatively ahead of the specification document. Building retry rules, escalation logic or delivery timing without a real specification to build them against would only create work that likely needs redoing once the actual requirements are known, so that section has deliberately been left exactly as is until the document is available.

Closing

The pattern across this milestone is worth stating plainly. Each layer of checking added this milestone found real problems the layer beneath it could not see. The automated test suite alone did not catch the booking defect, because it does not fire genuinely simultaneous requests. The database checks alone did not catch the broken images, because they do not load a real page in a real browser. And none of the earlier checks could have caught the server failing to start, because that only happens once the application is actually deployed somewhere real. Deploying now, rather than waiting until every other item was finished, is what surfaced twelve real problems while they were still cheap and safe to fix.